Lec-17: Operator Overloading in C++ continue..

Member vs. non-member signatures

```
// Member: left-hand side is *this
bool Student::operator<(
        const Student& rhs) const;

// Non-member: both sides are explicit parameters
bool operator<(const Student& lhs,
               const Student& rhs);
```

Non-member overloading is the form **preferred by the STL** and considered more idiomatic C++.

Why prefer non-member?

Two important advantages:

1. The left-hand side need not be your class.

A member overload forces the left operand to be an instance of the class. A non-member overload can accept *any* type on the left.

2. You can add operators to classes you don't own.

If you cannot edit the class definition (e.g. a library type), non-member overloading still lets you define comparisons between your type and the library type.

Rule of thumb: Use non-member overloading so that comparison works in *both* directions and across unrelated types.

The access problem

A non-member function lives *outside* the class. It has no `this` pointer and no access to private members:

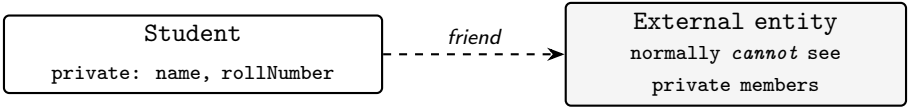
```
bool operator<(const Student& lhs,
               const Student& rhs)
{
    // ERROR: rollNumber is private
    return lhs.rollNumber < rhs.rollNumber;
}
```

Two solutions:

- Use only the public interface (getters).
- Declare the function as a **friend**.

What is `friend`?

A `friend` declaration inside a class grants a specific *external* entity direct access to the class's `private` and `protected` members.



That external entity can be one of two things:

- A **friend function** — a single free function or member function of another class.
- A **friend class** — an entire class, granting *all* of its member functions private access.

Why does C++ need `friend`?

Encapsulation says: keep data `private`, expose only a public interface. But sometimes a function or class that is *logically part of the type's interface* cannot be a member.

Without `friend`, we would be forced to either:

- Make fields `public` — breaking encapsulation entirely.
- Add getters for every field — cluttering the interface just for one external consumer.

Key idea: `friend` lets us keep data `private` while granting *surgical, controlled* access to the specific outsiders that need it.

The class is always the one granting access — an external entity cannot force its way in.

Friend function — syntax

A **friend function** is a free (non-member) function that is declared as a `friend` inside a class:

```
class MyClass {
private:
    int secret;
public:
    MyClass(int s) : secret(s) {}

    // grant access to this specific function
    friend void peek(const MyClass& obj);
};

// defined OUTSIDE the class -- not a member
void peek(const MyClass& obj) {
    std::cout << obj.secret << std::endl; // OK!
}
```

`peek` is *not* a member of `MyClass`. It has no `this` pointer. But it can read `obj.secret` because the class granted it friendship.

Friend function — the classic use case

The most common reason for a friend function: **non-member operator overloads**.

Consider `operator<<`:

- The left-hand operand is `std::ostream`, not our class.
- So the overload *must* be a non-member function.
- Yet it needs to read our private data to print it.

Without `friend`, we would need public getters for every field we want to print — even if no other part of the program needs those getters.

The same argument applies to any symmetric binary operator (`+`, `==`, `<`) where we prefer the non-member form for the reasons discussed earlier.

Friend function — another class's member

You can also befriend a *specific member function* of another class:

```
class Printer; // forward declaration

class Student {
private:
    std::string name;
    int rollNumber;
public:
    Student(std::string n, int r)
        : name(n), rollNumber(r) {}

    // only Printer::print can see our privates
    friend void Printer::print(const Student& s);
};
```

This is more restrictive than befriending the entire `Printer` class — only the single function `Printer::print` gets access.

Friend class — syntax

A **friend class** declaration grants *every* member function of that class access to private members:

```
class Student {
private:
    std::string name;
    int rollNumber;
public:
    Student(std::string n, int r)
        : name(n), rollNumber(r) {}

    friend class Registrar; // entire class is friend
};
```

Now *any* member function of `Registrar` can read and write `name` and `rollNumber` on any `Student` object — a broader grant than a single friend function.

Friend class — a real use case

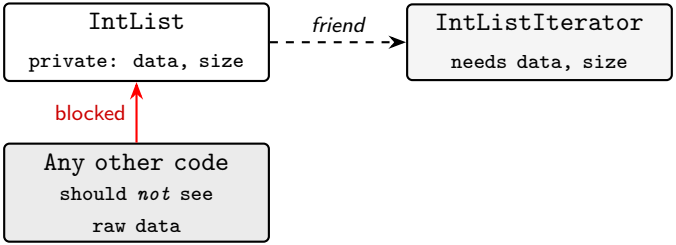
A classic pattern: a container grants friendship to its own iterator.

```
class IntList {
private:
    int* data;
    int size;
public:
    IntList(int* d, int s) : data(d), size(s) {}

    friend class IntListIterator;
};

class IntListIterator {
private:
    const IntList& list;
    int index;
public:
    IntListIterator(const IntList& l)
        : list(l), index(0) {}

    int current() const {
        return list.data[index]; // OK -- friend
    }
};
```

The iterator *must* see the raw `data` pointer, but no one else should.

Friend class — why not just use getters?

For the container/iterator example, exposing `data` and `size` through public getters would let *anyone* bypass the container's interface:



`friend class` lets us give the iterator full access while keeping everyone else locked out — precisely scoped trust.

Three properties of `friend`

Not symmetric

If A is a friend of B, B is *not* automatically a friend of A.

Not transitive

If A is a friend of B and B is a friend of C, A is *not* a friend of C.

Not inherited

If A is a friend of B, classes derived from B do *not* inherit the friendship.

These properties are deliberate design choices. Friendship is a *targeted, explicit grant* — it does not propagate automatically in any direction.

Friend function vs. friend class — summary

	Friend Function	**Friend Class**
Scope of access	One specific function	All member functions
Typical use	Operator overloads	Iterator / builder
Syntax	`friend void f();`	`friend class C;`
Precision	Narrow (preferred)	Broad (use sparingly)

Prefer the narrowest grant that works: if only one function needs access, make only that function a friend — not the whole class.

When to use — and when *not* to use — `friend`

Good uses:

- Non-member operator overloads that need private access (especially <<, >>, symmetric binary operators).
- A pair of tightly coupled classes where one implements the other's internals (e.g. an iterator for a container).

Warning signs:

- Declaring many unrelated functions or classes as friends — this is a sign that your encapsulation boundaries are wrong.
- Using `friend` simply to avoid writing getters — if a getter makes sense in the public interface, prefer that.

Rule of thumb: A `friend` declaration should feel like extending the class's own interface, not punching a hole through it.

Granting access with `friend`

Inside the class header, declare the non-member function as a friend:

```
class Student {
private:
    std::string name;
    int rollNumber;
public:
    Student(std::string name, int rollNumber);

    friend bool operator<(
        const Student& lhs,
        const Student& rhs);
};
```

The `friend` declaration grants the non-member function access to `private` members — without making it a member.

Implementation with `friend`

```
// NOT a member function -- no Student:: prefix
bool operator<(const Student& lhs,
               const Student& rhs)
{
    return lhs.rollNumber < rhs.rollNumber;
}
```

If you only use public getters, `friend` is not required:

```
bool operator<(const Student& lhs,
               const Student& rhs)
{
    return lhs.getRollNumber()
        < rhs.getRollNumber();
}
```

Ambiguity warning

```
Do not define both
a member operator<
and a non-member operator<
for the same pair of types.

The compiler cannot
resolve the ambiguity
⇒ undefined behaviour.
```

Pick **one** form and stick with it. For most operators, the non-member form with `friend` is the cleanest choice.

Passing arguments to operators

How should operator parameters be passed? The same rules as for any function, but the conventions are especially important here because operators are called *constantly*:

Object size	Convention	Example
Small (1–4 words)	Pass by value	`Fraction operator+(Fraction a, ...)`
Large	Pass by `const&`	`Matrix operator+(const Matrix\& a, ...)`
Will be modified	Pass by `&`	`ostream\& operator<<(ostream\& out, ...)`

The right-hand operand of a binary operator is almost always `const&` — we read it but do not modify it.

Returning results from operators

Operator kind	Return type	Why
Creates a new value	By value `Fraction operator+(...)`	New object; neither operand modified
Modifies `*this`	By reference `Fraction\& operator+=(...)`	Returns the modified object itself
Stream I/O	By reference `ostream\& operator<<(...)`	Returns the stream for chaining

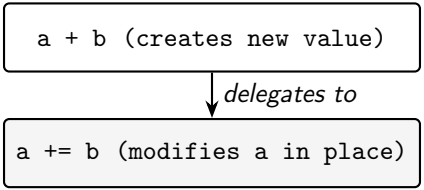
Never return a reference or pointer to a local variable — it will be destroyed when the function returns.

Define += first, then build + from it

Stroustrup's recommended pattern: define the **compound assignment** (+=) as a member, then define the **simple operator** (+) as a non-member that delegates to it:

```
class complex {
    double re, im;
public:
    // member: modifies *this, returns reference
    complex& operator+=(const complex& a) {
        re += a.re;
        im += a.im;
        return *this;
    }
};

// non-member: takes lhs BY VALUE, returns by value
complex operator+(complex a, const complex& b) {
    return a += b; // reuses += implementation
}
```

Why this pattern works



Advantages:

- **No code duplication** — the arithmetic logic lives in one place (`+=`).
- **Correct by construction** — `operator+` takes the left-hand side by *value*, so it works on a copy; the original is never modified.
- **Efficient** — `+=` avoids creating a temporary; the compiler can often elide the copy in `operator+`.
- **Consistency** — `a = a + b` and `a += b` are guaranteed to produce the same result.

Apply the same pattern to `-=`/`-`, `*=`/`*`, etc.

Principle of Least Astonishment (PoLA)

The behaviour of an operator should match what a reasonable programmer would expect.

Concretely:

- `operator+` should mean *addition* or *concatenation* — not subtraction.
- If the meaning is not obvious, write a named function instead.
- Operators communicate type semantics; misusing them destroys readability.

Rule of Contrariety

When you define one relational operator, define its logical complement in terms of it:

```
bool Student::operator==(
        const Student& other) const
{
    return (name == other.name)
        && (rollNumber == other.rollNumber);
}

bool Student::operator!=(
        const Student& other) const
{
    return !(*this == other);
}
```

This avoids duplication and guarantees that `==` and `!=` are always consistent.

Accessor functions: when getters make sense

- Stroustrup warns that providing individual get/set functions for every member is "an invitation to disaster" — it can compromise invariants and complicate future changes to the representation.

But accessors are appropriate when the parts have **independent semantic meaning**:

```
class complex {
public:
    constexpr double real() const { return re; }
    constexpr double imag() const { return im; }
    void real(double r) { re = r; }
    void imag(double i) { im = i; }
    // ...
};
```

Here `real()` and `imag()` reflect the mathematical structure of complex numbers — they are not just exposing implementation details.

Overloading operator«

The stream insertion operator is always a **non-member** because the left-hand operand is `std::ostream`, not our class:

```
std::ostream& operator<<(
        std::ostream& out,
        const Student& s)
{
    out << s.name << " (Roll #"
        << s.rollNumber << ")";
    return out;
}
```

Key design choices:

- Return `std::ostream&` to allow chaining (`std::cout << a << b`).
- Take `out` by *non-const* reference (we write to it).
- Take `s` by *const* reference (we only read it).

The 2 + z problem

Suppose `z` is a `complex`. We want both of these to compile:

```
complex r1 = z + 2; // complex + int
complex r2 = 2 + z; // int + complex
```

If `operator+` is a **member**, only the first works: `z.operator+(2)`. The second would require `2.operator+(z)` — but `2` is not a class object.

Solution: Make `operator+` a **non-member**. Both operands are then treated symmetrically by the compiler, and implicit conversions can apply to *either* side.

This is the deeper reason why non-member operators are preferred for symmetric binary operations.

Implicit conversions via constructors

How does `2 + z` actually work? The compiler finds:

```
complex operator+(complex a, const complex& b);
```

and a constructor that converts `double` to `complex`:

```
class complex {
public:
    complex(double r = 0, double i = 0)
        : re(r), im(i) {}
    // ...
};
```

So `2 + z` becomes `operator+(complex(2), z)` — the compiler inserts the conversion automatically.

Key rule: Only *one* level of user-defined implicit conversion is applied. If the conversion requires two steps, the compiler will reject it.

Many overloads or one converting constructor?

Two strategies for mixed-mode arithmetic:

Explicit overloads:

```
complex operator+(complex, complex);
complex operator+(complex, double);
complex operator+(double, complex);
// repeat for -, *, / ...
```

3 overloads per operator $\times$ 4 operators $=$ 12 functions.

Rely on conversion:

```
// single converting constructor
complex(double r = 0,
        double i = 0);

// one overload per operator
complex operator+(complex, complex);
```

The compiler inserts `complex(2)` for us.

Trade-off: explicit overloads can avoid conversion overhead and allow specialised fast paths. Relying on the constructor is simpler and scales better as types grow. For most types, the conversion approach is preferable.

`explicit` constructors: controlling conversions

A single-argument constructor doubles as an implicit conversion. Sometimes that is *not* what you want:

```
class Vector {
public:
    Vector(int size); // allocates size elements
};

void f(Vector v);
f(42); // compiles! Creates a 42-element Vector
       // almost certainly a bug
```

The fix: mark the constructor `explicit`:

```
class Vector {
public:
    explicit Vector(int size);
};

f(42); // ERROR: no implicit conversion
f(Vector(42)); // OK: explicit construction
```

Guideline: make single-argument constructors `explicit` by default. Remove `explicit` only when you *deliberately* want implicit conversion (as with `complex(double)`).

Conversion operator: `operator T()`

A constructor converts *to* your type. A **conversion operator** converts *from* your type to another:

```
class Tiny {
    char v;
public:
    Tiny(int i) { // int -> Tiny
        if (i < 0 || i > 63) throw Bad_range{};
        v = i;
    }

    operator int() const { return v; } // Tiny -> int
};
```

Now `Tiny` mixes freely with `int`:

```
Tiny a = 2;
Tiny b = 62;
int i = a + b; // a and b converted to int, i = 64
```

Note: The conversion operator has no explicit return type in its declaration — the target type is part of the operator name.

`explicit` conversion operators

Implicit conversions can cause surprises. The `explicit` keyword restricts when the conversion is applied:

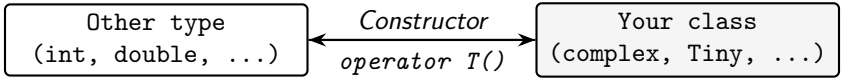
```
class SmartPtr {
public:
    explicit operator bool() const {
        return ptr != nullptr;
    }
private:
    int* ptr;
};
```

```
SmartPtr p = ...;
if (p) { ... } // OK: explicit context (if/while)
bool b = p; // ERROR: not explicit context
int x = p + p; // ERROR: bool not applied
```

Rule of thumb: always declare conversion operators `explicit` unless you specifically want implicit use. This avoids accidental conversions that silently compile but produce wrong results.

Conversions — choosing wisely

You have two conversion mechanisms. They serve opposite directions:



Stroustrup's advice:

- Prefer constructors for "into my type" conversions.
- Use `operator T()` sparingly and `explicit`ly.
- Having *both* user-defined conversions and user-defined operators on the same type easily creates ambiguities — prefer one mechanism or the other.
- If in doubt, start with a named function (e.g. `to_int()`) and promote it to a conversion operator only if the usage pattern justifies it.

`Fraction` Example

`+` addition	`==` equality	`<` ordering	`«` printing

Before we start: What should $\frac{1}{2} + \frac{1}{3}$ produce? What about $\frac{2}{4} == \frac{1}{2}$?
Think about the invariants we need.

Step 1: The class declaration (Fraction.h)

```cpp
#ifndef FRACTION_H
#define FRACTION_H

#include <iostream>

class Fraction {
private:
    int num; // numerator
    int den; // denominator (always > 0)

    void reduce(); // simplify to lowest terms
public:
    Fraction(int num = 0, int den = 1);

    int getNum() const;
    int getDen() const;

    // --- compound assignment (member) ---
    Fraction& operator+=(const Fraction& rhs);

    // --- relational (member) ---
    bool operator==(const Fraction& rhs) const;
    bool operator!=(const Fraction& rhs) const;
    bool operator<(const Fraction& rhs) const;

    // --- non-member friends ---
    friend Fraction operator+(
        Fraction lhs, const Fraction& rhs);
    friend std::ostream& operator<<(
        std::ostream& out, const Fraction& f);
};
#endif
```

Step 2: Constructor and `reduce()`

```
#include "Fraction.h"
#include <cstdlib> // abs

// Euclidean algorithm for GCD
static int gcd(int a, int b) {
    a = std::abs(a);
    b = std::abs(b);
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

void Fraction::reduce() {
    int g = gcd(num, den);
    num /= g;
    den /= g;
    if (den < 0) { num = -num; den = -den; }
}

Fraction::Fraction(int num, int den)
    : num(num), den(den) { reduce(); }
```

Invariant: after construction, `den > 0` and the fraction is in lowest terms. Every mutating path must call `reduce()`.

Step 3: operator+= then operator+

Predict first: What should `Fraction(1,2) + Fraction(1,3)` return?

First, the compound assignment (member — modifies `*this`):

```
Fraction& Fraction::operator+=(const Fraction& rhs) {
    num = num * rhs.den + rhs.num * den;
    den = den * rhs.den;
    reduce(); // restore invariant
    return *this; // return reference
}
```

Then, the simple operator (non-member — delegates to `+=`):

```
Fraction operator+(Fraction lhs, const Fraction& rhs) {
    return lhs += rhs; // lhs is a COPY (by value)
}
```

This is Stroustrup's recommended pattern: define `+=` once, get `+` for free with no code duplication.

Step 4: Relational operators

```
bool Fraction::operator==(
        const Fraction& rhs) const
{
    // Both sides already in lowest terms
    return (num == rhs.num) && (den == rhs.den);
}

bool Fraction::operator!=(
        const Fraction& rhs) const
{
    return !(*this == rhs); // rule of contrariety
}

bool Fraction::operator<(
        const Fraction& rhs) const
{
    // Cross-multiply (den is always positive)
    return num * rhs.den < rhs.num * den;
}
```

Note: The "always reduced" invariant makes `operator==` trivial — no cross-multiplication needed.

Step 5: operator« (non-member friend)

```
std::ostream& operator<<(std::ostream& out,
                        const Fraction& f)
{
    out << f.num << "/" << f.den;
    return out;
}
```

Why non-member? The left-hand side is `std::ostream`, not `Fraction`.

Why `friend`? We access `f.num` and `f.den` directly. Without `friend` we would need getters:

```
// Also works -- friend not needed
out << f.getNum() << "/" << f.getDen();
```

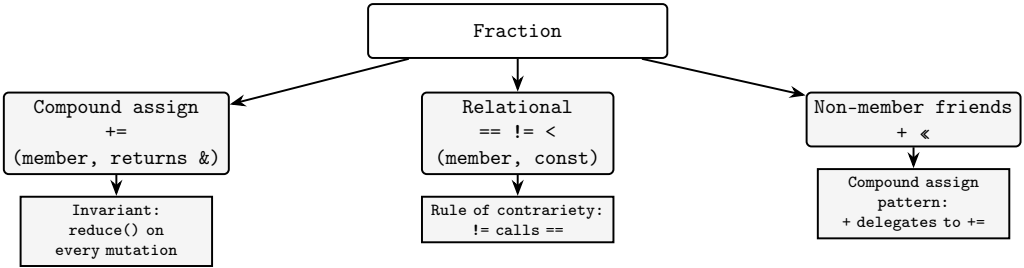
Step 6: Putting it together (main.cpp)

```cpp
#include "Fraction.h"
#include <iostream>
#include <algorithm> // std::min

int main() {
    Fraction a(1, 2);
    Fraction b(1, 3);

    std::cout << a << " + " << b
              << " = " << (a + b) << std::endl;

    a += b;
    std::cout << "After a += b: " << a << std::endl;

    Fraction c(2, 4);
    std::cout << c << " == " << Fraction(1,2)
              << " ? " << (c == Fraction(1,2))
              << std::endl;

    std::cout << "min(" << b << ", " << Fraction(1,4)
              << ") = " << std::min(b, Fraction(1,4))
              << std::endl;
    return 0;
}
```

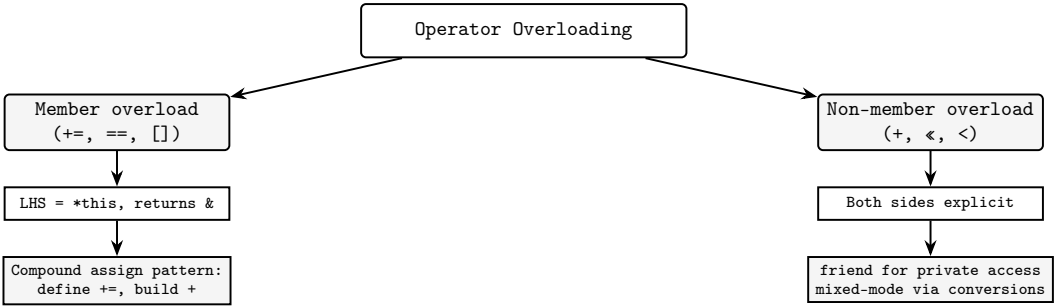
Expected output — predict before running:

```
1/2 + 1/3 = 5/6
After a += b:  5/6
1/2 == 1/2 ?  1
min(1/3, 1/4) = 1/4
```

Fraction Example



Operator overloading – the big picture



Questions?

References

1. R. Fernandez and T. Poimenidis, "Lecture 12: Operator Overloading," *CS106L — Standard C++ Programming*, Stanford University, Fall 2025.
   `https://web.stanford.edu/class/archive/cs/cs106l/cs106l.1262/lectures/2025Fall-12-OperatorOverloading.pdf`

2. F. Busato, "Object-Oriented Programming II — Polymorphism and Operator Overloading," *Modern C++ Programming*, 2026.
   `https://federico-busato.github.io/Modern-CPP-Programming/htmls/10.Object_Oriented_II.html`

3. B. Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley, 2013. Chapters 18–19.

4. "Operator Overloading," *cppreference.com*.
   `https://en.cppreference.com/w/cpp/language/operators`